

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

FILIPPE ROCHA MASCARENHAS

SISTEMA DE GERENCIAMENTO DE PADARIA

**CAMPOS DO JORDÃO
2026**

FILIPPE ROCHA MASCARENHAS

SISTEMA DE GERENCIAMENTO DE PADARIA

Projeto desenvolvido como requisito parcial para avaliação na disciplina de Banco de Dados 2 do curso de Tecnologia em Análise e Desenvolvimento de Sistemas, correspondente à entrega final do projeto sobre o desenvolvimento de um banco de dados não relacional (NoSQL).

Orientador: Paulo Giovani de Faria Zeferino

CAMPOS DO JORDÃO

2026

RESUMO

O presente documento detalha a migração e implementação de um banco de dados não relacional (NoSQL) voltado para um Sistema de Gerenciamento de Padarias, evoluindo a arquitetura de um modelo relacional prévio. O objetivo principal do trabalho consiste em estruturar uma base de dados orientada a documentos utilizando o MongoDB, visando maior escalabilidade horizontal, flexibilidade de esquema (*schema-less*) e alta performance em consultas de leitura para o controle transacional de vendas, gestão de estoque e categorização de produtos. A metodologia baseou-se na modelagem via *embedding* (documentos aninhados) e referências, substituindo tabelas tradicionais por coleções BSON. Os resultados obtidos demonstram a consolidação das operações por meio de coleções otimizadas, reduzindo a necessidade de junções complexas. Adicionalmente, o estudo apresenta a formulação de trinta consultas empregando o *Aggregation Pipeline*, extraíndo métricas estratégicas, como faturamento por período e rastreabilidade de estoque. O trabalho conclui que a adoção do MongoDB otimiza o fluxo de caixa rápido característico do varejo alimentício, validando a aplicação prática dos conceitos fundamentados na disciplina de Banco de Dados 2.

Palavras-Chave: Banco de dados NoSQL; MongoDB; Orientado a documentos; Gerenciamento de padarias; Aggregation Pipeline.

ABSTRACT

The present document details the migration and implementation of a non-relational database (NoSQL) aimed at a Bakery Management System, evolving the architecture from a previous relational model. The main objective of the work consists in structuring an efficient document-oriented database using the MongoDB DBMS, meeting the needs for horizontal scalability, schema flexibility (schema-less), and high performance in read queries for the transactional control of sales, inventory management, and product categorization. The methodology addressed the gathering of business requirements based on operational routines and the conversion of former relational tables into BSON collections, employing embedding techniques (nested documents) and references. The obtained results demonstrate the consolidation of commercial operations into optimized collections, ensuring the consistency of organizational information without the need for costly relational joins. Additionally, the study presents the formulation of thirty complex queries employing the Aggregation Pipeline, extracting strategic metrics such as revenue by period and stock traceability. The work concludes that the proposed NoSQL modeling optimizes managerial decision-making and accelerates the fast cash flow characteristic of food retail, validating the practical application of the concepts founded in the discipline.

Keywords: NoSQL database; MongoDB; Document-oriented; Bakery management; Aggregation Pipeline.

LISTA DE ALGORITMOS

ALGORITMO 1 – Criação das Coleções e Inserção de Dados – DML (O autor, 2026)	15
ALGORITMO 2 – Alerta de Reposição: Produtos Abaixo do Estoque Mínimo (O autor, 2026)	18
ALGORITMO 3 – Relatório de Exposição ao Crédito: Clientes com Fiado Ativo (O autor, 2026)	19
ALGORITMO 4 – Produtividade de Funcionários Ativos no Chão de Fábrica (O autor, 2026)	20
ALGORITMO 5 – Contatos de Fornecedores para Insumos em Baixa (O autor, 2026)	21
ALGORITMO 6 – Margem de Contribuição e Lucro Realizado por Produto (O autor, 2026)	22
ALGORITMO 7 – View Analítica: Custo Real de Fabricação por Produto (O autor, 2026)	23
ALGORITMO 8 – INNER JOIN Múltiplo: Detalhamento de Todas as Transações (O autor, 2026)	24
ALGORITMO 9 – Anti-join: Clientes sem Nenhuma Compra Registrada (O autor, 2026)	25
ALGORITMO 10 – Produtos Sem Lote de Produção Registrado (O autor, 2026)	26
ALGORITMO 11 – FULL OUTER JOIN: Fornecedores e Todos os Insumos (O autor, 2026)	26
ALGORITMO 12 – GROUP BY com Múltiplas Funções de Agregação por Pagamento (O autor, 2026)	27

ALGORITMO 13 – GROUP BY com HAVING: Preço Médio por Categoria (O autor, 2026)	29
ALGORITMO 14 – Rastreabilidade: Fornecedor, Insumo, Produto e Funcionário (O autor, 2026)	29
ALGORITMO 15 – Lógica Condicional IF/ELSE: Meta Diária de Produção (O autor, 2026)	31
ALGORITMO 16 – CASE WHEN: Classificação de Status Operacional do Estoque (O autor, 2026)	31
ALGORITMO 17 – Projeção de Consumo por Hora (Equivalente ao WHILE) (O autor, 2026)	32
ALGORITMO 18 – Transação Atômica: Reajuste de Preço com Rollback (O autor, 2026)	33
ALGORITMO 19 – Parâmetro de Saída: Total de Clientes com Fiado Ativo (O autor, 2026)	34
ALGORITMO 20 – UDF Escalar: Formatação de Preços em Moeda Brasileira (O autor, 2026)	35
ALGORITMO 21 – UDF Table-Valued: Relatório de Vendas por Funcionário (O autor, 2026)	35
ALGORITMO 22 – Auditoria com Registro de Exclusão (OUTPUT DELETED) (O autor, 2026)	36
ALGORITMO 23 – Atualização com Retorno Completo (OUTPUT INSERTED) (O autor, 2026)	37
ALGORITMO 24 – Trigger de Baixa Automática de Estoque Após Venda (O autor, 2026)	38

ALGORITMO 25 – UNION ALL: Consolidação do Cadastro de Pessoas e Empresas (O autor, 2026)	39
ALGORITMO 26 – INTERSECT: Produtos que Foram Vendidos e Produzidos (O autor, 2026)	40
ALGORITMO 27 – EXCEPT: Insumos sem Nenhuma Ficha Técnica Associada (O autor, 2026)	41
ALGORITMO 28 – Subconsulta com IN: Funcionários que Venderam e Produziram (O autor, 2026)	42
ALGORITMO 29 – EXISTS: Categorias com Produtos em Estoque Positivo (O autor, 2026)	43
ALGORITMO 30 – ANY/SOME: Preço de Venda Acima do Custo da Categoria Padaria (O autor, 2026)	44
ALGORITMO 31 – ALL: Produto com Preço de Venda Acima do Custo de TODOS (O autor, 2026)	45

SUMÁRIO

1	INTRODUÇÃO _____	9
1.1	Objetivos _____	9
1.2	Justificativa _____	10
1.3	Aspectos Metodológicos _____	10
1.4	Aporte Teórico _____	10
2	FUNDAMENTAÇÃO TEÓRICA _____	11
2.1	O Movimento NoSQL e o Teorema CAP _____	11
2.2	Modelos de Dados Não Relacionais _____	11
2.3	O SGBD MongoDB e o Armazenamento de Documentos ____	12
3	METODOLOGIA _____	13
3.1	Considerações Iniciais _____	13
3.2	Análise de Requisitos e Regras de Negócio _____	13
3.3	Ferramentas e Diretrizes de Modelagem NoSQL _____	14
4	RESULTADOS OBTIDOS _____	15
4.1	Estrutura Lógica das Coleções _____	15
4.2	Script de Inserção de Dados (DML) _____	15
4.3	Execução e Discussão das 30 Consultas NoSQL _____	18
5	CONCLUSÃO _____	46
	REFERÊNCIAS _____	47

1 INTRODUÇÃO

Neste capítulo são introduzidos os assuntos abordados por este documento, apresentando a motivação, o cenário abordado, os objetivos e a organização do texto. O cenário refere-se ao setor de panificação e confeitaria, caracterizado por um fluxo diário intenso de transações, alta rotatividade de estoque perecível e modalidades distintas de atendimento. O modelo de dados relacional atende a parte dessas demandas, porém a necessidade de alta disponibilidade, evolução constante do catálogo de produtos e consultas rápidas sem junções pesadas (*JOINS*) torna os bancos de dados NoSQL alternativas modernas e altamente eficientes. Motivada por essa realidade tecnológica, a presente pesquisa utiliza o MongoDB como ativo estratégico para estruturar as informações transacionais de um Sistema de Gerenciamento de Padarias de forma flexível e escalável.

1.1 Objetivos

Este trabalho tem por objetivo geral projetar, estruturar e implementar um banco de dados não relacional orientado a documentos, garantindo a performance transacional de vendas e o monitoramento eficiente de estoque em um ambiente de panificação.

Para a consecução do objetivo principal, foram estabelecidos os seguintes objetivos específicos:

- Levantar e adaptar os requisitos operacionais essenciais do negócio para um modelo não relacional;
- Estudar os conceitos teóricos da arquitetura NoSQL;
- Modelar as coleções de dados utilizando técnicas de *embedding* (aninhamento) e referências;
- Implementar o esquema físico no SGBD MongoDB;
- Formular trinta consultas reais utilizando a linguagem de consulta do MongoDB (MQL) e o *Aggregation Pipeline*.

1.2 Justificativa

A justificativa deste documento dá-se pela necessidade de aplicar os conceitos de bancos de dados modernos (NoSQL) adquiridos na disciplina de Banco de Dados 2 do IFSP - Câmpus Campos do Jordão. Adicionalmente, estruturar o sistema em um modelo orientado a documentos justifica-se pela aderência dessa tecnologia às aplicações web e mobile modernas, permitindo a persistência direta de objetos JSON e garantindo agilidade na manutenção do sistema de vendas.

1.3 Aspectos Metodológicos

O presente estudo fez uso de pesquisa bibliográfica para fundamentar os conceitos de NoSQL e de modelagem prática para a codificação das coleções e consultas no MongoDB, baseando-se na transposição de um projeto relacional previamente estruturado.

1.4 Aporte Teórico

As bases teóricas que sustentam esta investigação fundamentam-se nos princípios estabelecidos por Chodorow (2013) e Sadalage e Fowler (2012) sobre o movimento NoSQL e os mecanismos de escalabilidade horizontal. A seguir, na seção 2, detalha-se a fundamentação conceitual do paradigma de documentos. A seção 3 explicita as regras de modelagem e desnormalização aplicadas ao negócio. Na seção 4, realizou-se a exposição física do código e a avaliação crítica dos resultados das 30 consultas, finalizando com as conclusões e melhorias futuras na seção 5.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo apresenta-se a revisão da literatura técnica especializada sobre bancos de dados não relacionais, servindo como alicerce conceitual para o desenvolvimento do sistema proposto.

2.1 O Movimento NoSQL e o Teorema CAP

Os bancos de dados não relacionais, amplamente conhecidos sob a sigla NoSQL (*Not Only SQL*), surgiram como resposta às limitações de escalabilidade vertical e rigidez estrutural dos SGBDs relacionais clássicos frente ao crescimento massivo de aplicações web modernas. Conforme defendido por Sadalage e Fowler (2012), o modelo NoSQL abdica do esquema rígido de tabelas (*schema-less*) em prol de flexibilidade e distribuição de dados.

Diferente do ecossistema SQL que prioriza as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade), os sistemas NoSQL operam sob as premissas do Teorema CAP, formulado por Eric Brewer. O teorema postula que um sistema distribuído pode garantir apenas duas de três propriedades simultaneamente: Consistência (C), Disponibilidade (A) e Tolerância a Particionamento (P). No varejo de panificação, a alta disponibilidade de escrita no caixa é prioritária, justificando o uso de sistemas que toleram consistência eventual para manter o PDV sempre ativo.

2.2 Modelos de Dados Não Relacionais

O ecossistema NoSQL segmenta-se em quatro grandes categorias, especializadas de acordo com a forma de armazenamento e o padrão de acesso do sistema:

- Chave-Valor: O modelo mais simples, onde os dados são mapeados a partir de chaves exclusivas que apontam para valores brutos (Ex: Redis). Oferece latência mínima para leituras pontuais.

- Família de Colunas (Colunares): Organiza o armazenamento fisicamente em colunas em vez de linhas. Apresenta alta performance para agregações em grandes volumes de registros (Ex: Apache Cassandra).
- Grafos: Focado na persistência de entidades (nós) e em suas interconexões diretas (arestas), sendo ideal para redes complexas e sistemas de recomendação (Ex: Neo4j).
- Orientado a Documentos: Estrutura as informações em unidades semiestruturadas autossuficientes denominadas documentos, comumente nos formatos JSON, XML ou BSON. É o modelo escolhido para este estudo devido à facilidade de mapear objetos de software de forma direta.

2.3 O SGBD MongoDB e o Armazenamento de Documentos

O MongoDB é consolidado como o SGBD líder na categoria orientada a documentos. Conforme explicado por Chodorow (2013), ele armazena dados em formato BSON (*Binary JSON*), uma extensão binária do JSON que suporta tipos de dados adicionais como datas e dados binários.

A grande vantagem teórica do MongoDB sobre o modelo de tabelas consiste no suporte a documentos aninhados (*embedding*). Informações que no SQL exigiriam divisões e chaves estrangeiras para evitar redundâncias, no MongoDB podem ser salvas em forma de subdocumentos ou listas (*arrays*) dentro de um único registro principal. Isso otimiza drasticamente as operações de leitura, pois o SGBD recupera todo o agregado de dados de uma entidade realizando uma única operação de I/O no disco.

3 METODOLOGIA

Nesta seção serão apresentadas detalhadamente a metodologia utilizada neste trabalho, as razões para as escolhas arquitetônicas orientadas a documentos e as etapas seguidas para a concepção e migração do Sistema de Gerenciamento de Padarias. O capítulo aborda as considerações iniciais sobre o escopo do projeto, a análise de requisitos baseada em rotinas comerciais reais e, por fim, as ferramentas tecnológicas e diretrizes adotadas para a modelagem NoSQL.

3.1 Considerações Iniciais

A concepção metodológica deste projeto visa remodelar as operações de uma padaria a partir de um paradigma moderno de persistência. A escolha pelo modelo NoSQL do MongoDB baseia-se na necessidade de gerenciar um catálogo dinâmico de produtos com rapidez de caixa e alta tolerância a falhas na ponta de venda, integrando os dados de vendas com o controle de produção fabril.

3.2 Análise de Requisitos e Regras de Negócio

O mapeamento das regras de negócio apoiou-se no estudo e na vivência prática de rotinas comerciais do setor de panificação, tomando como base o fluxo operacional prático vivenciado na Padoka do Alemão. A partir da análise dos processos de atendimento, venda a prazo e controle diário de fornadas, traduziram-se os requisitos operacionais em regras que guiaram a modelagem lógica não relacional:

- RD01 (Cadastro de Produtos): Armazenamento de dados cadastrais flexíveis (suportando itens com ou sem código de barras).
- RD02 (Categorização): Agrupamento dinâmico sem a necessidade de tabelas associativas rígidas.
- RD03 e RD04 (Controle de Vendas): Registro imediato do cupom fiscal, encapsulando os itens da venda e congelando o preço histórico praticado no momento da compra.

- RD06 (Exposição ao Crédito): Cadastro de clientes autorizados a utilizar a modalidade de pagamento "Fiado", atrelando o limite de crédito permitido.
- RD07 (Rastreabilidade do Chão de Fábrica): Registro de lotes de produção associados à composição de insumos.

3.3 Ferramentas e Diretrizes de Modelagem NoSQL

Para o desenvolvimento do banco de dados não relacional, utilizou-se o MongoDB Atlas para hospedagem do cluster em nuvem e a interface gráfica MongoDB Compass integrada ao terminal Mongosh para codificação dos scripts.

Diferente da abordagem relacional focada na normalização rígida (3FN), a metodologia NoSQL baseou-se nas seguintes diretrizes de design de documentos:

- Aninhamento (Embedding): Aplicado para relações do tipo 1:N onde os dados filhos pertencem estritamente ao pai e são consultados em conjunto. Os "Itens da Venda" foram embutidos como uma lista de subdocumentos na coleção de vendas, e a "Ficha Técnica" tornou-se um array de insumos dentro da própria coleção de produtos. Isso elimina operações de junção e garante atomicidade local.
- Referenciamento (Referencing): Aplicado para relações onde os dados existem de forma independente e crescem indefinidamente. Os documentos da coleção vendas guardam apenas os IDs de referência de clientes e funcionarios, evitando duplicações massivas de dados que sofrem alterações frequentes (como o endereço do cliente ou cargo do funcionário).

4 RESULTADOS OBTIDOS

Neste capítulo são detalhados os resultados práticos obtidos com a migração, apresentando o desenho lógico das coleções, o script completo de carga e a execução comentada das trinta consultas gerenciais.

4.1 Estrutura Lógica das Coleções

Como resultado da aplicação das técnicas de desnormalização, as antigas 10 tabelas relacionais do sistema de padarias foram consolidadas em 5 coleções estruturadas no MongoDB, descritas a seguir através da notação JSON:

- Coleção clientes: Armazena dados cadastrais e o limite de crédito.
- Coleção funcionarios: Registra a equipe e situação atual.
- Coleção fornecedores: Mantém o contato corporativo para matérias-primas.
- Coleção produtos (Agrega Categoria, Insumos e Ficha Técnica via Embedding):
- Coleção vendas (Agrega os Itens de Venda via Embedding e referência Cliente/Funcionário):

4.2 Script de Inserção de Dados (DML)

O algoritmo abaixo detalha a carga inicial aplicada ao banco. A base de dados foi meticulosamente construída para conter os cenários exigidos pelas consultas (como ausência de campos, limites de crédito zerados e estoques críticos).

```
1 db.funcionarios.insertMany([
2   { _id: 1, nome_completo: "Carlos Silva", cpf: "111.222.333-44",
3   cargo: "Caixa", situacao: "Ativo" },
4   { _id: 2, nome_completo: "Ana Pereira", cpf: "555.666.777-88",
5   cargo: "Padeiro Líder", situacao: "Ativo" },
6   { _id: 3, nome_completo: "Roberto Souza", cpf: "999.888.777-66",
7   cargo: "Gerente", situacao: "Férias" },
```

```

8      { _id: 4, nome_completo: "Mariana Costa", cpf: "444.555.666-77",
9      cargo: "Atendente", situacao: "Ativo" }
10  });
11
12  db.clientes.insertMany([
13      { _id: 1, nome_completo: "Maria Oliveira", cpf_cnpj:
14      "123.456.789-00", telefone: "12988887777", limite_fiado: 250.00 },
15      { _id: 2, nome_completo: "João Fernandes", cpf_cnpj:
16      "098.765.432-11", telefone: "12977776666", limite_fiado: 0.00 },
17      { _id: 3, nome_completo: "Padaria Express", cpf_cnpj:
18      "44.555.666/0001-77", telefone: "1233334444", limite_fiado: 1000.00
19  },
20      { _id: 4, nome_completo: "Lucas Almeida", cpf_cnpj:
21      "111.999.888-22", telefone: "11955554444", limite_fiado: 50.00 },
22      { _id: 5, nome_completo: "Fernanda Lima", cpf_cnpj:
23      "222.333.444-55", telefone: "11944443333", limite_fiado: 0.00 }
24  ]);
25
26  db.fornecedores.insertMany([
27      { _id: 1, razao_social: "Moinho S/A", cnpj: "11.111.111/0001-11",
28      telefone: "1133334444" },
29      { _id: 2, razao_social: "Laticínios Vale", cnpj:
30      "22.222.222/0001-22", telefone: "1234567890" },
31      { _id: 3, razao_social: "Distribuidora Café", cnpj:
32      "33.333.333/0001-33", telefone: "11999998888" },
33      { _id: 4, razao_social: "Embalagens Brasil", cnpj:
34      "44.444.444/0001-44", telefone: "1122223333" }
35  ]);
36
37  db.produtos.insertMany([
38      { _id: 1, nome_produto: "Pão Francês", preco_custo: 0.35,
39      preco_venda: 1.00, estoque_atual: 500, estoque_minimo: 100,
40      categoria: { nome: "Padaria" }, ficha_tecnica: [{ insumo: "Farinha",
41      qtd: 0.05, id_forn: 1 }] },
42      { _id: 2, nome_produto: "Bolo de Cenoura", preco_custo: 12.00,
43      preco_venda: 35.00, estoque_atual: 10, estoque_minimo: 5, categoria:
44      { nome: "Confeitaria" }, ficha_tecnica: [{ insumo: "Farinha", qtd:
45      0.3, id_forn: 1 }, { insumo: "Leite", qtd: 0.2, id_forn: 2 }] },
46      { _id: 3, nome_produto: "Café Expresso", preco_custo: 1.50,
47      preco_venda: 6.00, estoque_atual: 150, estoque_minimo: 20, categoria:
48      { nome: "Bebidas" }, ficha_tecnica: [{ insumo: "Grão", qtd: 0.02,
49      id_forn: 3 }] },
50      { _id: 4, nome_produto: "Croissant", preco_custo: 2.50,
51      preco_venda: 8.00, estoque_atual: 8, estoque_minimo: 15, categoria: {
52      nome: "Padaria" }, ficha_tecnica: [{ insumo: "Farinha", qtd: 0.1,
53      id_forn: 1 }, { insumo: "Manteiga", qtd: 0.05, id_forn: 2 }] },
54      { _id: 5, nome_produto: "Queijo Prato 100g", preco_custo: 3.50,
55      preco_venda: 5.50, estoque_atual: 4, estoque_minimo: 10, categoria: {
56      nome: "Frios" } } ],

```



```

57     { _id: 6, nome_produto: "Suco de Laranja", preco_custo: 4.00,
58 preco_venda: 9.00, estoque_atual: 30, estoque_minimo: 10, categoria:
59 { nome: "Bebidas" } },
60     { _id: 7, nome_produto: "Torta de Morango", preco_custo: 25.00,
61 preco_venda: 65.00, estoque_atual: 2, estoque_minimo: 3, categoria: {
62 nome: "Confeitaria" } }
63   ]);
64
65   db.vendas.insertMany([
66     { _id: 1, data_hora: ISODate("2026-06-25T08:30:00Z"),
67 forma_pagamento: "Dinheiro", id_funcionario: 1, id_cliente: null,
68 itens: [{ id_produto: 1, qtd: 10, preco: 1.00 }, { id_produto: 3,
69 qtd: 2, preco: 6.00 }], total_venda: 22.00 },
70     { _id: 2, data_hora: ISODate("2026-06-25T09:15:00Z"),
71 forma_pagamento: "Fiado", id_funcionario: 4, id_cliente: 1, itens: [{
72 id_produto: 2, qtd: 1, preco: 35.00 }, { id_produto: 4, qtd: 4,
73 preco: 8.00 }], total_venda: 67.00 },
74     { _id: 3, data_hora: ISODate("2026-06-26T14:20:00Z"),
75 forma_pagamento: "PIX", id_funcionario: 1, id_cliente: 3, itens: [{
76 id_produto: 1, qtd: 50, preco: 0.90 }, { id_produto: 5, qtd: 5,
77 preco: 5.50 }], total_venda: 72.50 },
78     { _id: 4, data_hora: ISODate("2026-06-26T16:45:00Z"),
79 forma_pagamento: "Cartão", id_funcionario: 4, id_cliente: 2, itens:
80 [{ id_produto: 7, qtd: 1, preco: 65.00 }], total_venda: 65.00 },
81     { _id: 5, data_hora: ISODate("2026-06-27T07:10:00Z"),
82 forma_pagamento: "Dinheiro", id_funcionario: 1, id_cliente: 4, itens:
83 [{ id_produto: 1, qtd: 5, preco: 1.00 }, { id_produto: 6, qtd: 1,
84 preco: 9.00 }], total_venda: 14.00 },
85     { _id: 6, data_hora: ISODate("2026-06-27T18:00:00Z"),
86 forma_pagamento: "Cartão", id_funcionario: 4, id_cliente: null,
87 itens: [{ id_produto: 3, qtd: 1, preco: 6.00 }], total_venda: 6.00 }
88   ]);
89
90   db.producao.insertMany([
91     { _id: 1, data_hora: ISODate("2026-06-25T05:00:00Z"), qtd: 300,
92 obs: "Manhã", id_produto: 1, id_funcionario: 2 },
93     { _id: 2, data_hora: ISODate("2026-06-25T14:00:00Z"), qtd: 200,
94 obs: "Tarde", id_produto: 1, id_funcionario: 2 },
95     { _id: 3, data_hora: ISODate("2026-06-26T06:00:00Z"), qtd: 15, obs:
96 "Bolos diários", id_produto: 2, id_funcionario: 2 },
97     { _id: 4, data_hora: ISODate("2026-06-27T05:30:00Z"), qtd: 40, obs:
98 "Croissants", id_produto: 4, id_funcionario: 2 }
99   ]);

```

Algoritmo 1 - Criação das Coleções e Inserção de Dados (DML)

(O autor, 2026)

4.3 Execução e Discussão das 30 Consultas NoSQL

Nesta subseção são apresentadas as trinta consultas MQL (MongoDB Query Language) correspondentes as trinta consultas SQL do projeto relacional. Cada consulta mantém a equivalência lógica com sua contraparte relacional, demonstrando que o banco de dados NoSQL é plenamente capaz de responder às mesmas perguntas gerenciais com o mesmo conjunto de dados. As consultas utilizam os dados exatos inseridos no DML: 3 funcionários, 3 clientes, 3 fornecedores, 5 produtos, 4 vendas, 6 itens de venda e 3 lotes de produção.

Consulta 01 – Alerta de Reposição: Produtos Abaixo do Estoque Mínimo com Categoria Ativa

Descrição: Reproduz a lógica da Consulta SQL 1, identificando produtos cujo estoque atual está abaixo de 1,5 vezes o estoque mínimo e cuja categoria está marcada como ativa. Utiliza \$expr para comparar campos internos ao documento e retorna a folga de estoque em ordem crescente, replicando o comportamento do INNER JOIN + WHERE do modelo relacional.

```
1 db.produtos.aggregate([
2   { $match: {
3     "categoria.ativa": true,
4     $expr: { $lte: ["$estoque_atual",
5       { $multiply: ["$estoque_minimo", 1.5] } ] }
6   }},
7   { $project: {
8     _id: 0,
9     nome_produto: 1,
10    "categoria.nome": 1,
11    cod_barra: { $ifNull: ["$cod_barra", "SemCodigo"] },
12    folga: { $subtract: ["$estoque_atual", "$estoque_minimo"] }
13  }},
14   { $sort: { folga: 1 } }
15 ])
```

Algoritmo 2 – Consulta 01 – Alerta de Reposição: Produtos Abaixo do Estoque Mínimo com Categoria Ativa (O autor, 2026)

Saída:

nome_produto	categoria.nome	cod_barra	folga

Bolo de Cenoura	Confeitaria	7891000000002	8
Queijo Prato	Frios	7891000000005	10
Croissant	Padaria	7891000000004	20

Consulta 02 – Relatório de Exposição ao Crédito: Clientes com Fiado Ativo

Descrição: Espelha a Consulta SQL 2, calculando o total já comprado no modo Fiado por clientes com limite maior que zero. O \$lookup com pipeline interno filtra apenas as vendas com forma_pagamento Fiado e soma os itens. Clientes com limite ativo mas sem compras no fiado retornam zero, replicando o LEFT JOIN + ISNULL do SQL.

```
1 db.clientes.aggregate([
2   { $match: { limite_fiado: { $gt: 0 } } },
3   { $lookup: {
4     from: "vendas",
5     let: { cid: "$_id" },
6     pipeline: [
7       { $match: { $expr: { $and: [
8         { $eq: ["$_id_cliente", "$$cid"] },
9         { $eq: ["$forma_pagamento", "Fiado"] }
10        ] } } },
11       { $unwind: "$itens" },
12       { $group: { _id: null,
13         total_fiado: { $sum: {
14           $multiply: ["$itens.quantidade",
15             "$itens.preco_unit_aplicado"]
16         } } } }
17     ],
18     as: "compras_fiado"
19   } },
20   { $project: { _id: 0, nome_completo: 1, cpf_cnpj: 1,
21     limite_fiado: 1,
22     total_comprado_fiado: {
23       $ifNull: [{ $arrayElemAt: ["$compras_fiado.total_fiado", 0] }, 0]
24     }
25   } }
26 ])
```

Algoritmo 3 – Consulta 02 – Relatório de Exposição ao Crédito: Clientes com Fiado Ativo (O autor, 2026)

Saída:

nome_completo	cpf_cnpj	limite	total_fiado

Maria Oliveira	123.456.789-00	250	72.5
Padaria Express	44.555.666/0001-77	1000	0

Consulta 03 – Produtividade de Funcionários Ativos no Chão de Fabrica

Descrição: Corresponde a Consulta SQL 3. Filtrar funcionários com situação Ativo e cargo contendo Padeiro, e realizar \$lookup na coleção produção para contar lotes e somar volume total produzido. Replica o LEFT JOIN + GROUP BY + COUNT + SUM do modelo relacional, confirmando que Ana Pereira é a única padeira ativa responsável pelos 3 lotes.

```
1 db.funcionarios.aggregate([
2   { $match: {
3     situacao: "Ativo",
4     cargo: { $regex: "Padeiro", $options: "i" }
5   }},
6   { $lookup: {
7     from: "producao",
8     localField: "_id",
9     foreignField: "id_funcionario",
10    as: "lotes"
11  }},
12  { $project: {
13    _id: 0, nome_completo: 1, cargo: 1,
14    lotes_produzidos: { $size: "$lotes" },
15    volume_total: { $sum: "$lotes.quantidade_produzida" }
16  }}
17 ])
```

Algoritmo 4 – Consulta 03 – Produtividade de Funcionários Ativos no Chão de Fabrica (O autor, 2026)

Saída:

nome_completo	cargo	lotes_prod	volume_total
---------------	-------	------------	--------------

Ana Pereira		Padeiro Lider		3 260

Consulta 04 – Contatos de Fornecedores para Insumos em Baixa

Descrição: Réplica a Consulta SQL 4. Expande as fichas técnicas dos produtos, filtra insumos com estoque abaixo de 100 unidades e resolve o fornecedor responsável via \$lookup, retornando razão social, e-mail e dados do insumo. Identifica que Grao de Cafe (50 KG) e Manteiga (30 KG) demandam reposição urgente.

```

1 db.produtos.aggregate([
2   { $unwind: "$ficha_tecnica" },
3   { $match: {
4     "ficha_tecnica.estoque_insumo": { $lt: 100 }
5   }},
6   { $lookup: {
7     from: "fornecedores",
8     localField: "ficha_tecnica.id_fornecedor",
9     foreignField: "_id",
10    as: "forn"
11  }},
12  { $project: {
13    _id: 0,
14    razao_social: { $arrayElemAt: ["$forn.razao_social",0] },
15    email: { $arrayElemAt: ["$forn.email",0] },
16    nome_insumo: "$ficha_tecnica.nome_insumo",
17    estoque_atual:"$ficha_tecnica.estoque_insumo",
18    unidade: "$ficha_tecnica.unidade"
19  }},
20  { $sort: { estoque_atual: 1 } }
21 ])
```

Algoritmo 5 – Consulta 04 – Contatos de Fornecedores para Insumos em Baixa (O autor, 2026)

Saída:

razao_social		email		insumo		estq		un

Distribuidora Cafe		cafe@distribuidora.com		Grao de Cafe		50		KG
Laticinios Vale		contato@vale.com.br		Manteiga		30		KG

Consulta 05 – Margem de Contribuição e Lucro Realizado por Produto

Descrição: Espelha a Consulta SQL 5. Para cada produto calcular o lucro bruto unitário (preco_venda - preco_custo), a quantidade total vendida e o lucro realizado pelo preço histórico de cada venda via \$lookup com pipeline. Produtos sem vendas retornam zero via \$ifNull.

```
1 db.produtos.aggregate([
2   { $lookup: {
3     from: "vendas",
4     let: { pid: "$_id", custo: "$preco_custo" },
5     pipeline: [
6       { $unwind: "$itens" },
7       { $match: {
8         $expr:{$eq:["$itens.id_produto","$$pid"]} } },
9       { $group: { _id: null,
10        qtd: { $sum: "$itens.quantidade" },
11        lucro: { $sum: {
12          $multiply: ["$itens.quantidade",
13            {$subtract:["$itens.preco_unit_aplicado","$$custo"]}]}
14        }}}}
15   ],
16   as: "vinfo"
17 },
18 { $project: { _id:0, nome_produto:1,
19   lucro_unit:{$subtract:["$preco_venda","$preco_custo"]},
20   qtd_vendida:{$ifNull:[$arrayElemAt:["$vinfo.qtd",0]],0}},
21   lucro_realizado:{$ifNull:[$arrayElemAt:["$vinfo.lucro",0]],0}}
22 },
23 { $sort: { lucro_realizado: -1 } }
24 ])
```

Algoritmo 6 – Consulta 05 – Margem de Contribuição e Lucro Realizado por Produto (O autor, 2026)

Saída:

nome_produto	lucro_unit	qtd_vendida	lucro_realizado

Pao Frances	0.65	65	37.25
Croissant	5.00	5	25.00
Bolo de Cenoura	23.00	1	23.00

<i>Queijo Prato</i>	20.00	0.5	10.00
<i>Cafe Expresso</i>	4.50	2	9.00

Consulta 06 – View Analítica: Custo Real de Fabricacao por Produto

Descrição: Equivalente a Consulta SQL 6 (CREATE VIEW vw_Custo_Fabricacao). O pipeline expande a ficha técnica de cada produto e calcula o custo real de insumos (quantidade_utilizada x custo_unitario embarcado), agrupando por produto.

```

1 db.produtos.aggregate([
2   { $match: {
3     ficha_tecnica: { $exists: true,
4                       $not: { $size: 0 } }
5   }},
6   { $unwind: "$ficha_tecnica" },
7   { $group: {
8     _id: { id: "$_id", nome: "$nome_produto" },
9     custo_real: {
10      $sum: {
11        $multiply: [
12          "$ficha_tecnica.qtd_utilizada",
13          "$ficha_tecnica.custo_unitario"
14        ]
15      }
16    }
17  }},
18  { $project: {
19    _id: 0,
20    id_produto: "$_id.id",
21    nome_produto: "$_id.nome",
22    custo_real: { $round: ["$custo_real", 6] }
23  }},
24  { $sort: { id_produto: 1 } }
25 ])
```

Algoritmo 7 – Consulta 06 – View Analítica: Custo Real de Fabricação por Produto (O autor, 2026)

Saída:

<i>id_produto</i>	<i>nome_produto</i>	<i>custo_real</i>

1	Pao Frances	0.175
2	Bolo de Cenoura	1.89
3	Cafe Expresso	0.9
4	Croissant	1.6

Consulta 07 – INNER JOIN Múltiplo: Detalhamento de Todas as Transações

Descrição: Corresponde a Consulta SQL 7 (INNER JOIN entre Venda, Funcionario, Item_Venda e Produto). Realiza dois \$lookup para resolver funcionario e produto e \$unwind nos itens, listando cada linha de venda com o operador, o produto e o total do item. Confirma que Carlos Silva é o único operador em todas as 4 vendas.

```

1 db.vendas.aggregate([
2   { $unwind: "$itens" },
3   { $lookup: {
4     from: "funcionarios",
5     localField: "id_funcionario",
6     foreignField: "_id",
7     as: "func"
8   }},
9   { $lookup: {
10    from: "produtos",
11    localField: "itens.id_produto",
12    foreignField: "_id",
13    as: "prod"
14  }},
15  { $project: {
16    _id: 0,
17    id_venda: "$_id",
18    operador: { $arrayElemAt: ["$func.nome_completo",0] },
19    produto: { $arrayElemAt: ["$prod.nome_produto",0] },
20    total_item: { $round: [{
21      $multiply: ["$itens.quantidade",
22        "$itens.preco_unit_aplicado"]
23    },2] }
24  }},
25  { $sort: { id_venda: 1 } }
26 ])
```

Algoritmo 8 – Consulta 07 – INNER JOIN Múltiplo: Detalhamento de Todas as Transações
(O autor, 2026)

Saída:

<i>id_venda</i>	<i>operador</i>	<i>produto</i>	<i>total_item</i>

1	Carlos Silva	Pao Frances	15
1	Carlos Silva	Cafe Expresso	12
2	Carlos Silva	Bolo de Cenoura	35
2	Carlos Silva	Croissant	37.5
3	Carlos Silva	Pao Frances	45
4	Carlos Silva	Queijo Prato	27.5

Consulta 08 – Anti-join: Clientes sem Nenhuma Compra Registrada

Descrição: Reproduz a Consulta SQL 8 (LEFT OUTER JOIN com WHERE IS NULL). Faz \$lookup de clientes para vendas e filtra apenas documentos onde o array resultante está vazio. Com os dados do projeto, todos os clientes possuem ao menos uma venda, retornando zero documentos.

```
1 db.clientes.aggregate([
2   { $lookup: {
3     from: "vendas",
4     localField: "_id",
5     foreignField: "id_cliente",
6     as: "compras"
7   }},
8   { $match: { compras: { $size: 0 } } },
9   { $project: {
10     _id:0, nome_completo:1, telefone:1
11   }}
12 ])
```

Algoritmo 9 – Consulta 08 – Anti-join: Clientes sem Nenhuma Compra Registrada (O autor, 2026)

Saída:

<i>nome_completo</i> <i>telefone</i>

(0 documentos retornados)

Consulta 09 – Produtos Sem Lote de Produção Registrado

Descrição: Equivalente a Consulta SQL 9 (RIGHT OUTER JOIN). O anti-join inverso é obtido fazendo \$lookup de produtos para produção e filtrando onde o array de lotes está vazio. Identifica Café Expresso e Queijo Prato como produtos sem nenhum lote de produção registrado.

```
1 db.produtos.aggregate([
2   { $lookup: {
3     from: "producao",
4     localField: "_id",
5     foreignField: "id_produto",
6     as: "lotes"
7   }},
8   { $match: { lotes: { $size: 0 } } },
9   { $project: { _id:0, nome_produto:1 } }
10 ])
```

Algoritmo 10 – Consulta 09 – Produtos Sem Lote de Produção Registrado (O autor, 2026)

Saída:

nome_produto

Cafe Expresso
Queijo Prato

Consulta 10 – FULL OUTER JOIN: Fornecedores e Todos os Insumos

Descrição: Reproduz a Consulta SQL 10 (FULL OUTER JOIN entre Fornecedor e Insumo). Simulado expandindo fichas técnicas via \$lookup e \$unwind com preserveNullAndEmptyArrays, garantindo que todos os fornecedores aparecem mesmo sem insumos associados.

```
1 db.fornecedores.aggregate([
2   { $lookup: {
3     from: "produtos",
4     let: { fid: "$_id" },
```

```

5     pipeline: [
6       { $unwind: "$ficha_tecnica" },
7       { $match: { $expr: {
8         $eq:["$ficha_tecnica.id_fornecedor","$$fid"] } } },
9       { $project: {
10        nome_insumo: "$ficha_tecnica.nome_insumo",
11        _id:0 } }
12     ],
13     as: "insumos"
14  }},
15  { $unwind: {
16    path: "$insumos",
17    preserveNullAndEmptyArrays: true
18  }},
19  { $project: {
20    _id: 0,
21    empresa: { $ifNull: ["$razao_social","Sem Fornecedor"] },
22    material: { $ifNull: ["$insumos.nome_insumo","Nenhum"] }
23  }}
24 ]))

```

Algoritmo 11 – Consulta 10 – FULL OUTER JOIN: Fornecedores e Todos os Insumos (O autor, 2026)

Saída:

<i>empresa</i>	<i> material</i>

<i>Moinho S/A</i>	<i> Farinha de Trigo</i>
<i>Laticinios Vale</i>	<i> Leite Integral</i>
<i>Laticinios Vale</i>	<i> Manteiga</i>
<i>Distribuidora Cafe</i>	<i> Grao de Cafe</i>

Consulta 11 – GROUP BY com Múltiplas Funções de Agregação por Forma de Pagamento

Descrição: Espelha a Consulta SQL 11. Desempilha os itens de cada venda, agrupa por forma_pagamento e calcula faturamento total (\$sum), ticket médio (\$avg) e maior item (\$max). Os valores reproduzem exatamente a saída do modelo relacional.

```

1 db.vendas.aggregate([

```

```

2  { $unwind: "$itens" },
3  { $addFields: {
4      valor_item: {
5          $multiply: ["$itens.quantidade", "$itens.preco_unit_aplicado"]
6      }
7  }},
8  { $group: {
9      _id: "$forma_pagamento",
10     fat_total: { $sum: "$valor_item" },
11     ticket_medio: { $avg: "$valor_item" },
12     maior_venda: { $max: "$valor_item" }
13 }},
14 { $project: {
15     _id: 0,
16     forma_pagamento: "$_id",
17     fat_total: { $round: ["$fat_total", 2] },
18     ticket_medio: { $round: ["$ticket_medio", 2] },
19     maior_venda: { $round: ["$maior_venda", 2] }
20 }},
21 { $sort: { fat_total: -1 } }
22 ]}

```

Algoritmo 12 – Consulta 11 – GROUP BY com Múltiplas Funções de Agregação por Forma de Pagamento (O autor, 2026)

Saída:

<i>forma_pagamento</i>	<i>fat_total</i>	<i>ticket_medio</i>	<i>maior_venda</i>

<i>Fiado</i>	72.5	36.25	37.5
<i>PIX</i>	45	45	45
<i>Cartao</i>	27.5	27.5	27.5
<i>Dinheiro</i>	27	13.5	15

Consulta 12 – GROUP BY com HAVING: Preço Médio por Categoria Acima de R\$ 2,00

Descrição: Corresponde a Consulta SQL 12. Agrupar produtos por categoria embarcada, calcula o preço médio de venda e filtra via \$match (equivalente ao HAVING) categorias cujo preço médio supera R\$2,00. Todas as quatro categorias ativas superam o limite.

```

1 db.produtos.aggregate([
2   { $group: {
3     _id: "$categoria.nome",
4     preco_medio: { $avg: "$preco_venda" }
5   }},
6   { $match: { preco_medio: { $gt: 2.00 } } },
7   { $project: {
8     _id: 0,
9     nome_categoria: "$_id",
10    preco_medio: { $round: ["$preco_medio",2] }
11  }},
12  { $sort: { preco_medio: 1 } }
13])

```

Algoritmo 13 – Consulta 12 – GROUP BY com HAVING: Preço Médio por Categoria Acima de R\$ 2,00 (O autor, 2026)

Saída:

<i>nome_categoria</i>	<i>preco_medio</i>

<i>Padaria</i>	<i>4.25</i>
<i>Bebidas</i>	<i>6</i>
<i>Confeitaria</i>	<i>35</i>
<i>Frios</i>	<i>55</i>

Consulta 13 – Rastreabilidade : Fornecedor, Insumo, Produto e Funcionário

Descrição: Réplica a Consulta SQL 13, que encadeia cinco JOINS. Três \$lookup são encadeados: o primeiro resolve o produto produzido, o segundo o funcionário responsável e o terceiro os fornecedores dos insumos da ficha técnica, gerando a cadeia completa de rastreabilidade.

```

1 db.producao.aggregate([
2   { $lookup: {
3     from: "produtos",
4     localField: "id_produto",
5     foreignField: "_id",
6     as: "produto"
7   }},
8   { $lookup: {
9     from: "funcionarios",

```

```

10     localField: "id_funcionario",
11     foreignField: "_id",
12     as: "funcionario"
13   }},
14   { $unwind: "$produto" },
15   { $unwind: "$produto.ficha_tecnica" },
16   { $lookup: {
17     from: "fornecedores",
18     localField: "produto.ficha_tecnica.id_fornecedor",
19     foreignField: "_id",
20     as: "fornecedor"
21   }},
22   { $project: {
23     _id: 0,
24     fornecedor:   {$arrayElemAt: ["$fornecedor.razao_social", 0]},
25     insumo:       "$produto.ficha_tecnica.nome_insumo",
26     produto_final: "$produto.nome_produto",
27     produzido_por: {$arrayElemAt: ["$funcionario.nome_completo", 0]}
28   }}
29 ]})

```

Algoritmo 14 – Consulta 13 – Rastreabilidade: Fornecedor, Insumo, Produto e Funcionário
(O autor, 2026)

Saída:

<i>fornecedor</i>	<i>insumo</i>	<i>produto</i>	<i>funcionario</i>

<i>Moinho S/A</i>	<i>Farinha de Trigo</i>	<i>Pao Frances</i>	<i>Ana Pereira</i>
<i>Moinho S/A</i>	<i>Farinha de Trigo</i>	<i>Bolo de Cenoura</i>	<i>Ana Pereira</i>
<i>Laticinios Vale</i>	<i>Leite Integral</i>	<i>Bolo de Cenoura</i>	<i>Ana Pereira</i>
<i>Moinho S/A</i>	<i>Farinha de Trigo</i>	<i>Croissant</i>	<i>Ana Pereira</i>
<i>Laticinios Vale</i>	<i>Manteiga</i>	<i>Croissant</i>	<i>Ana Pereira</i>

Consulta 14 – Lógica Condicional IF/ELSE: Verificação da Meta Diária de Produção

Descrição: Corresponde a Consulta SQL 14 (DECLARE + IF...ELSE). A lógica condicional é implementada via \$cond: se o volume total produzido atingir ou superar 50 unidades, retorna mensagem de meta atingida; caso contrário, exibe alerta. Os 260 itens produzidos superam a meta.

```

1 db.producao.aggregate([
2   { $group: {
3     _id: null,
4     total: { $sum: "$quantidade_produzida" }
5   }},
6   { $project: {
7     _id: 0,
8     total: 1,
9     status_meta: {
10      $cond: {
11        if: { $gte: ["$total",50] },
12        then: { $concat: ["Meta Atingida: ",
13          { $toString: "$total" } ] },
14        else: { $concat: ["Abaixo da Meta: ",
15          { $toString: "$total" } ] }
16      }
17    }
18  }}
19 ])
```

Algoritmo 15 – Consulta 14 – Lógica Condicional IF/ELSE: Verificação da Meta Diária de Produção (O autor, 2026)

Saída:

<i>total</i> <i>status_meta</i>

260 Meta Atingida: 260

Consulta 15 – CASE WHEN: Classificação de Status Operacional do Estoque

Descrição: Espelha a Consulta SQL 15 (CASE WHEN). Utiliza \$switch com múltiplos branches: estoque igual a 0 resulta em Crítico, abaixo de 10 em Alerta, e demais em Normal. Com os dados do projeto, todos os produtos estão com estoque adequado.

```

1 db.produtos.aggregate([
2   { $project: {
3     _id: 0, nome_produto: 1, estoque_atual: 1,
4     status: {
5       $switch: {
6         branches: [
7           { case: { $eq: ["$estoque_atual",0] },
```

```

8         then: "Critico - Interromper Vendas" },
9         { case: { $lt: ["$estoque_atual",10] },
10          then: "Alerta - Iniciar Producao" }
11     ],
12     default: "Normal - Estoque Adequado"
13 }
14 }
15 }}
16 ))

```

Algoritmo 16 – Consulta 15 – CASE WHEN: Classificação de Status Operacional do Estoque (O autor, 2026)

Saída:

<i>nome_produto</i>	<i>estoque_atual</i>	<i>status</i>

<i>Pao Frances</i>	<i>500</i>	<i>Normal - Estoque Adequado</i>
<i>Bolo de Cenoura</i>	<i>10</i>	<i>Normal - Estoque Adequado</i>
<i>Cafe Expresso</i>	<i>150</i>	<i>Normal - Estoque Adequado</i>
<i>Croissant</i>	<i>30</i>	<i>Normal - Estoque Adequado</i>
<i>Queijo Prato</i>	<i>15</i>	<i>Normal - Estoque Adequado</i>

Consulta 16 – Projeção de Consumo por Hora (Equivalente ao WHILE)

Descrição: Reproduz a lógica do laço WHILE da Consulta SQL 16, que simulava o consumo hora a hora do estoque de pão Francês. A iteração é gerada com \$range, criando um array de horas e calculando o estoque projetado para cada intervalo de 50 unidades.

```

1 db.produtos.aggregate([
2   { $match: { _id: 1 } },
3   { $project: {
4     _id: 0,
5     horas: { $range: [1,4] },
6     estoque_inicial: "$estoque_atual"
7   }},
8   { $unwind: "$horas" },
9   { $project: {
10     projecao_hora: {
11       $concat: ["Hora ", { $toString: "$horas" }]
12     },

```



```

13     estoque_projetado: {
14         $subtract: ["$estoque_inicial",
15                     { $multiply: ["$horas",50] }]
16     }
17 }
18 ]

```

Algoritmo 17 – Consulta 16 – Projeção de Consumo por Hora (Equivalente ao WHILE) (O autor, 2026)

Saída:

projecao_hora estoque_projetado	

Hora 1	450
Hora 2	400
Hora 3	350

Consulta 17 – Transação Atômica: Reajuste de Preço por Categoria com Rollback

Descrição: Corresponde a Consulta SQL 17 (Stored Procedure com TRY/CATCH e BEGIN TRANSACTION). Transações ACID multi-documento são realizadas com Session. O script aplica reajuste de 10% nos preços da categoria Padaria. Em caso de erro, executa abortTransaction, equivalente ao ROLLBACK.

```

1 // Executar no mongosh
2 const session = db.getMongo().startSession();
3 session.startTransaction();
4 try {
5     const col = session
6         .getDatabase("padaria").produtos;
7     col.updateMany(
8         { "categoria.nome": "Padaria" },
9         { $mul: { preco_venda: 1.10 } },
10        { session }
11    );
12    session.commitTransaction();
13    print("Reajuste aplicado com sucesso.");
14 } catch (e) {
15     session.abortTransaction();
16     print("Transacao revertida: " + e.message);

```

```

17 } finally {
18     session.endSession();
19 }

```

Algoritmo 18 – Consulta 17 – Transação Atômica: Reajuste de Preço por Categoria com Rollback (O autor, 2026)

Saída:

```

Reajuste aplicado com sucesso.
(Pao Frances: 1.00 -> 1.10 | Croissant: 7.50 -> 8.25)

```

Consulta 18 – Parâmetro de Saída: Total de Clientes com Fiado Ativo

Descrição: Replica a Consulta SQL 18 (Stored Procedure com parâmetro OUTPUT). Uma função JavaScript conta os clientes com limite_fiado maior que zero e retorna o valor em uma variável. Resultado confirma que há 2 clientes autorizados: Maria Oliveira e Padaria Express.

```

1 // Executar no mongosh
2 function contarClientesFiado() {
3     return db.clientes.countDocuments(
4         { limite_fiado: { $gt: 0 } }
5     );
6 }
7
8 const total = contarClientesFiado();
9 print("Clientes com Fiado Ativo: " + total);

```

Algoritmo 19 – Consulta 18 – Parâmetro de Saída: Total de Clientes com Fiado Ativo (O autor, 2026)

Saída:

```

Clientes com Fiado Ativo: 2

```

Consulta 19 – UDF Escalar: Formatação de Preços em Moeda Brasileira

Descrição: Equivalente a Consulta SQL 19 (CREATE FUNCTION fn_FormatarMoedaBR). A UDF escalar é simulada com \$concat e \$toString dentro

de um \$project, formatando o preço de venda com o prefixo R\$. Os preços refletem o estado após o reajuste da Consulta 17.

```
1 db.produtos.aggregate([
2   { $project: {
3     _id: 0,
4     nome_produto: 1,
5     preco_pdv: {
6       $concat: [
7         "R$ ",
8         { $toString: { $round: ["$preco_venda",2] } }
9       ]
10    }
11  }}
12 ])
```

Algoritmo 20 – Consulta 19 – UDF Escalar: Formatação de Preços em Moeda Brasileira (O autor, 2026)

Saída:

nome_produto	preco_pdv

Pao Frances	R\$ 1.1
Bolo de Cenoura	R\$ 35
Cafe Expresso	R\$ 6
Croissant	R\$ 8.25
Queijo Prato	R\$ 55

Consulta 20 – UDF Table-Valued: Relatorio de Vendas por Funcionario

Descrição: Reproduz a Consulta SQL 20 (CREATE FUNCTION fn_RelatorioVendasFuncionario). O pipeline recebe id_funcionario como parâmetro, faz \$lookup para resolver o cliente e retorna todas as vendas do funcionário. Executado para id_funcionario = 1 (Carlos Silva), retorna às 4 vendas.

```
1 // Para id_funcionario = 1 (Carlos Silva)
2 db.vendas.aggregate([
3   { $match: { id_funcionario: 1 } },
4   { $lookup: {
5     from: "clientes",
```

```

6      localField: "id_cliente",
7      foreignField: "_id",
8      as: "cliente"
9    }},
10   { $project: {
11     _id: 0,
12     id_venda: "$_id",
13     data_hora: 1,
14     forma_pagamento: 1,
15     cliente: {
16       $ifNull: [{ $arrayElemAt: ["$cliente.nome_completo", 0] }, null]
17     }
18   }},
19   { $sort: { id_venda: 1 } }
20 ]))

```

Algoritmo 21 – Consulta 20 – UDF Table-Valued: Relatório de Vendas por Funcionário (O autor, 2026)

Saída:

<i>id</i>	<i>data_hora</i>	<i>pagamento</i>	<i>cliente</i>

1	2026-05-28T10:48:23Z	Dinheiro	null
2	2026-05-28T10:48:23Z	Fiado	Maria Oliveira
3	2026-05-28T10:48:23Z	PIX	Padaria Express
4	2026-05-28T10:48:23Z	Cartao	Joao Fernandes

Consulta 21 – Auditoria com Registro de Exclusão (Equivalente ao OUTPUT DELETED)

Descrição: Equivalente a Consulta SQL 21 (DELETE com OUTPUT DELETED). A auditoria de exclusões é feita com `findOneAndDelete`, que retorna o documento removido. O registro é inserido em uma coleção de auditoria com `data_exclusao`. Executado com Cliente Teste, retorna null pois não existe.

```

1 const removido = db.clientes.findOneAndDelete(
2   { nome_completo: "Cliente Teste" }
3 );
4
5 if (removido) {
6   db.auditoria_clientes.insertOne({

```

```

7     ...removido,
8     data_exclusao: new Date()
9   });
10   print("Auditado: " + removido.nome_completo);
11 } else {
12   print("(0 documentos afetados)");
13 }

```

Algoritmo 22 – Consulta 21 – Auditoria com Registro de Exclusão (Equivalente ao OUTPUT DELETED) (O autor, 2026)

Saída:

```
(0 documentos afetados)
```

Consulta 22 – Atualizacao com Retorno Completo (Equivalente ao OUTPUT INSERTED/DELETED)

Descrição: Corresponde a Consulta SQL 22 (UPDATE com OUTPUT). findOneAndUpdate com returnDocument captura o documento antes e depois da alteração. O script reduz o estoque de Farinha de Trigo em 10 unidades e exibe os valores anterior e atualizado, replicando a saída 1000 a 990.

```

1 const antes = db.produtos.findOne(
2   { "ficha_tecnica.nome_insumo": "Farinha de Trigo" },
3   { "ficha_tecnica.$": 1 }
4 );
5 const depois = db.produtos.findOneAndUpdate(
6   { "ficha_tecnica.nome_insumo": "Farinha de Trigo" },
7   { $inc: { "ficha_tecnica.$.estoque_insumo": -10 } },
8   { returnDocument: "after" }
9 );
10 print("Anterior:   " +
11   antes.ficha_tecnica[0].estoque_insumo);
12 print("Atualizado: " +
13   depois.ficha_tecnica[0].estoque_insumo);

```

Algoritmo 23 – Consulta 22 – Atualizacao com Retorno Completo (Equivalente ao OUTPUT INSERTED/DELETED) (O autor, 2026)

Saída:

```
Anterior:   1000
```

```
Atualizado: 990
```

Consulta 23 – Trigger de Baixa Automática de Estoque Após Venda

Descrição: Reproduz a Consulta SQL 23 (CREATE TRIGGER trg_Baixa_Estoque_Venda). No MongoDB, Change Streams monitoram a coleção de vendas. Quando uma nova venda é inserida, o handler decrementa o estoque de cada produto. O teste insere 5 Cafés Expressos e confirma a baixa de 150 para 145.

```
1 print("Estoque ANTES:");
2 printjson(db.produtos.findOne(
3   { _id: 3 },
4   { nome_produto:1, estoque_atual:1 }
5 ));
6
7 // Simula handler do Change Stream
8 const itens = [
9   { id_produto:3, quantidade:5, preco_unit_aplicado:6.00 }
10 ];
11 itens.forEach(item => {
12   db.produtos.updateOne(
13     { _id: item.id_produto },
14     { $inc: { estoque_atual: -item.quantidade } }
15   );
16 });
17 print("Estoque DEPOIS:");
18 printjson(db.produtos.findOne(
19   { _id: 3 },
20   { nome_produto:1, estoque_atual:1 }
21 ));
```

Algoritmo 24 – Consulta 23 – Trigger de Baixa Automática de Estoque Após Venda (O autor, 2026)

Saída:

<i>Estoque ANTES:</i>
<i>{ _id: 3, nome_produto: 'Cafe Expresso', estoque_atual: 150 }</i>
<i>Estoque DEPOIS:</i>
<i>{ _id: 3, nome_produto: 'Cafe Expresso', estoque_atual: 145 }</i>

Consulta 24 – UNION ALL: Consolidação do Cadastro de Pessoas e Empresas

Descrição: Espelha a Consulta SQL 24 (UNION ALL entre Cliente, Fornecedor e Funcionario). A união é obtida com \$unionWith, que concatena resultados de múltiplas coleções em um único pipeline. Cada coleção projeta os campos no mesmo schema (tipo_relacao, nome_razao, contato).

```
1 db.clientes.aggregate([
2   { $project: { _id:0,
3     tipo_relacao: { $literal:"Cliente" },
4     nome_razao: "$nome_completo",
5     contato: { $ifNull: ["$telefone","N/A"] } } },
6   { $unionWith: { coll: "fornecedores", pipeline: [
7     { $project: { _id:0,
8       tipo_relacao: { $literal:"Fornecedor" },
9       nome_razao: "$razao_social",
10      contato: "$cnpj" } }
11   ]}},
12   { $unionWith: { coll: "funcionarios", pipeline: [
13     { $project: { _id:0,
14       tipo_relacao: { $literal:"Funcionario" },
15       nome_razao: "$nome_completo",
16       contato: "$cargo" } }
17   ]}},
18   { $sort: { tipo_relacao:1, nome_razao:1 } }
19 ])
```

Algoritmo 25 – Consulta 24 – UNION ALL: Consolidação do Cadastro de Pessoas e Empresas (O autor, 2026)

Saída:

tipo_relacao	nome_razao	contato

Cliente	Joao Fernandes	12977776666
Cliente	Maria Oliveira	12988887777
Cliente	Padaria Express	1233334444
Fornecedor	Distribuidora Cafe	33.333.333/0001-33
Fornecedor	Laticinios Vale	22.222.222/0001-22
Fornecedor	Moinho S/A	11.111.111/0001-11
Funcionario	Ana Pereira	Padeiro Lider
Funcionario	Carlos Silva	Caixa
Funcionario	Roberto Souza	Gerente

Consulta 25 – INTERSECT: Produtos que Foram Vendidos e Produzidos

Descrição: Corresponde a Consulta SQL 25 (INTERSECT entre Item_Venda e Producao). A intersecção é obtida com \$setIntersection entre os ids dos produtos vendidos e produzidos. O resultado confirma que Pão Francês, Bolo de Cenoura e Croissant aparecem em ambas as coleções.

```
1 db.vendas.aggregate([
2   { $unwind: "$itens" },
3   { $group: { _id: null,
4     ids_v: { $addToSet: "$itens.id_produto" } } },
5   { $lookup: {
6     from: "producao",
7     pipeline: [
8       { $group: { _id: null,
9         ids_p: { $addToSet: "$id_produto" } } }
10    ],
11    as: "prod"
12  }},
13  { $project: { _id: 0,
14    em_comum: { $setIntersection: [
15      "$ids_v",
16      { $arrayElemAt: ["$prod.ids_p", 0] }
17    ] }
18  }},
19  { $unwind: "$em_comum" },
20  { $project: { id_produto: "$em_comum" } },
21  { $sort: { id_produto: 1 } }
22 ])
```

Algoritmo 26 – Consulta 25 – INTERSECT: Produtos que Foram Vendidos e Produzidos (O autor, 2026)

Saída:

<i>id_produto</i>

1
2
4

Consulta 26 – EXCEPT/DIFFERENCE: Insumos sem Nenhuma Ficha Técnica Associada

Descrição: Reproduz a Consulta SQL 26 (EXCEPT entre Insumo e Ficha_Técnica). A diferença de conjuntos é obtida via \$setDifference entre todos os ids de insumos e os presentes nas fichas técnicas. Como todos os 4 insumos possuem ficha, o resultado retorna zero documentos.

```
1 db.produtos.aggregate([
2   { $unwind: "$ficha_tecnica" },
3   { $group: { _id: null,
4     c_ficha: { $addToSet:"$ficha_tecnica.id_insumo" } } },
5   { $lookup: {
6     from: "produtos",
7     pipeline: [
8       { $unwind: "$ficha_tecnica" },
9       { $group: { _id: null,
10         todos: { $addToSet:"$ficha_tecnica.id_insumo" } } }
11     ],
12     as: "t"
13   }},
14   { $project: { _id:0,
15     sem_ficha: { $setDifference: [
16       { $arrayElemAt: ["$t.todos",0] },
17       "$c_ficha"
18     ]}
19   }}
20 ])
```

Algoritmo 27 – Consulta 26 – EXCEPT/DIFFERENCE: Insumos sem Nenhuma Ficha Técnica Associada (O autor, 2026)

Saída:

<i>sem_ficha</i>

[] (0 insumos sem ficha tecnica associada)

Consulta 27 – Subconsulta com IN: Funcionários que Venderam e Produziram

Descrição: Corresponde a Consulta SQL 27 (IN com duas subconsultas correlacionadas). Os ids de funcionários que venderam e produziram são coletados em arrays separados e \$setIntersection encontra o cruzamento. Como Carlos só vende e Ana só produz, retorna zero documentos.

```
1 db.vendas.aggregate([
2   { $group: { _id: null,
3     ids_v: { $addToSet: "$id_funcionario" } } },
4   { $lookup: {
5     from: "producao",
6     pipeline: [
7       { $group: { _id: null,
8         ids_p: { $addToSet: "$id_funcionario" } } }
9     ],
10    as: "prod"
11  }},
12  { $project: { comuns: { $setIntersection: [
13    "$ids_v",
14    { $arrayElemAt: ["$prod.ids_p",0] }
15  ]}}},
16  { $unwind: "$comuns" },
17  { $lookup: {
18    from: "funcionarios",
19    localField: "comuns",
20    foreignField: "_id",
21    as: "func"
22  }},
23  { $project: { _id:0,
24    nome:{ $arrayElemAt: ["$func.nome_completo",0]},
25    cargo:{ $arrayElemAt: ["$func.cargo",0] } } }
26 ])
```

Algoritmo 28 – Consulta 27 – Subconsulta com IN: Funcionários que Venderam e Produziram (O autor, 2026)

Saída:

<i>nome cargo</i>
<i>-----</i>
<i>(0 documentos retornados)</i>

Consulta 28 – EXISTS: Categorias que Possuem Produtos com Estoque Positivo

Descrição: Reproduz a Consulta SQL 28 (WHERE EXISTS). O EXISTS é simulado com \$lookup e \$match que filtram categorias cujo array de produtos com estoque_atual > 0 seja não vazio. Todas as quatro categorias possuem ao menos um produto em estoque.

```
1 db.produtos.aggregate([
2   { $group: { _id: "$categoria.nome" } },
3   { $lookup: {
4     from: "produtos",
5     let: { cat: "$_id" },
6     pipeline: [
7       { $match: { $expr: { $and: [
8         { $eq: ["$categoria.nome", "$$cat"] },
9         { $gt: ["$estoque_atual", 0] }
10      ]}}}
11     ],
12     as: "em_estoque"
13   }},
14   { $match: { "em_estoque.0": { $exists: true } } },
15   { $project: { _id: 0, nome_categoria: "$_id" } },
16   { $sort: { nome_categoria: 1 } }
17 ])
```

Algoritmo 29 – Consulta 28 – EXISTS: Categorias que Possuem Produtos com Estoque Positivo (O autor, 2026)

Saída:

nome_categoria

Bebidas
Confeitaria
Frios
Padaria

Consulta 29 – ANY/SOME: Preço de Venda Acima do Custo de Qualquer Produto da Categoria Padaria

Descrição: Espelha a Consulta SQL 29 (WHERE preco_venda > ANY). O operador ANY é reproduzido com \$lookup que coleta os custos da categoria Padaria e \$match com \$gt sobre o \$min do array. Qualquer produto cujo preço supere ao menos um desses custos é retornado.

```
1 db.produtos.aggregate([
2   { $lookup: {
3     from: "produtos",
4     pipeline: [
5       { $match: { "categoria.nome": "Padaria" } },
6       { $group: { _id: null,
7         custos: { $push: "$preco_custo" } } }
8     ],
9     as: "ref"
10  }},
11  { $addFields: {
12    custos: { $arrayElemAt: ["$ref.custos",0] }
13  }},
14  { $match: { $expr: {
15    $gt: ["$preco_venda",{ $min: "$custos" }]
16  }}}},
17  { $project: { _id:0, nome_produto:1, preco_venda:1 } }
18 ])
```

Algoritmo 30 – Consulta 29 – ANY/SOME: Preço de Venda Acima do Custo de Qualquer Produto da Categoria Padaria (O autor, 2026)

Saída:

nome_produto	preco_venda

Pao Frances	1.1
Bolo de Cenoura	35
Cafe Expresso	6
Croissant	8.25
Queijo Prato	55

Consulta 30 – ALL: Produto com Preço de Venda Acima do Custo de TODOS os Produtos

Descrição: Corresponde a Consulta SQL 30 (WHERE preco_venda > ALL). O operador ALL é reproduzido com \$lookup que coleta todos os preços de custo e \$match com \$gt sobre o \$max. Apenas Queijo Prato (R\$55,00) supera o maior custo de todos (R\$35,00 do Bolo de Cenoura).

```
1 db.produtos.aggregate([
2   { $lookup: {
3     from: "produtos",
4     pipeline: [
5       { $group: { _id: null,
6         maior_custo: { $max: "$preco_custo" } } }
7     ],
8     as: "ref"
9   }},
10  { $addFields: {
11    maior_custo: { $arrayElemAt: ["$ref.maior_custo",0] }
12  }},
13  { $match: { $expr: {
14    $gt: ["$preco_venda","$maior_custo"]
15  } }},
16  { $project: { _id:0, nome_produto:1, preco_venda:1 } }
17 ])
```

Algoritmo 31 – Consulta 30 – ALL: Produto com Preço de Venda Acima do Custo de TODOS os Produtos (O autor, 2026)

Saída:

<i>nome_produto preco_venda</i>

<i>Queijo Prato 55</i>

5.CONCLUSÃO

O presente projeto alcançou pleno êxito em remodelar a estrutura de dados de um sistema integrado de panificação para o paradigma NoSQL, demonstrando os ganhos práticos da desnormalização sobre o modelo relacional clássico. A reestruturação lógica das informações provou que os objetivos de controle transacional veloz (RD03) e gerenciamento de estoque (RD04) são atingidos com maior performance computacional no MongoDB através da técnica de aninhamento (*embedding*) de documentos. Ao embutir os itens comercializados diretamente nas vendas e as composições técnicas nos produtos, erradicou-se a necessidade de junções de tabelas em tempo de execução, solucionando gargalos de I/O em momentos de fluxo intenso de caixa.

A bateria de trinta consultas formuladas utilizando a sintaxe MQL e pipelines de agregação comprovou que a flexibilidade de esquema (*schema-less*) não corrompe a capacidade analítica gerencial do sistema, viabilizando o cálculo de indicadores estratégicos (KPIs) cruciais como faturamento, ticket médio e curvas de perdas de insumos. Ademais, o ciclo metodológico fechou-se ao comprovar que a maleabilidade estrutural do MongoDB atendeu com precisão a todos os sete requisitos levantados no início do estudo.

Como sugestões para possíveis melhorias futuras que podem ser realizadas no escopo deste projeto, indica-se a implementação do mecanismo de transações multi-documento nativo do MongoDB Atlas para assegurar a atomicidade estrita nas rotinas onde a inserção de uma venda precisa obrigatoriamente decrementar o estoque físico de insumos de forma síncrona. Por fim, propõe-se o desenvolvimento de um microsserviço em *Node.js* conectado a uma interface gráfica responsiva em *React*, abstraindo os comandos JSON executados para entregar um ecossistema amigável de frente de caixa e controle de terminal fabril aos operadores da padaria.

REFERÊNCIAS

BANKER, Kyle. **MongoDB in Action**. 2. ed. Nova York: Manning Publications, 2016 .

CHODOROW, Kristina. **MongoDB: The Definitive Guide**. 2. ed. Sebastopol: O'Reilly Media, 2013 .

DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. 8. ed. Rio de Janeiro: Campus, 2004 .

SADALAGE, Pramod J.; FOWLER, Martin. **NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence**. 1. ed. Boston: Addison-Wesley, 2012.